

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1001	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	V. SHASHANK SAI	Reg. No.:	192524509
Date:	1-10-2026	Duration:	60 minutes

Code of Conduct

I V.SHASHANK SAI (192524509 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.SHASHANK SAI

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. Understand the Problem Statement

- Explain the problem in your own words.
- Identify the objectives of the proposed system.
- State the scope of the problem.

2. Identify Key Stakeholders and Challenges

- Identify all the stakeholders involved in the system.
- Explain the roles and responsibilities of each stakeholder.
- Discuss the major challenges and issues faced by the stakeholders.

3. Analyze the Current Approach

- Describe the existing system or current process.
- Identify its strengths and limitations.
- Analyze the problems or gaps that need to be addressed.

4. Propose a Suitable Solution Using Software Engineering Principles

- Design a software-based solution for the given problem.
- Describe the major functional modules of the proposed system.
- Identify the functional and non-functional requirements.
- Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.

5. Justify the Chosen Methodology/Framework

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing;

	strong technical justification.			report lacks organization and clarity.
--	---------------------------------	--	--	--

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Problem Statement

AI-Powered Crop Yield Prediction and Optimization

1. Understand the Problem Statement

1.1 Problem Description

Crop yield depends on several connected factors, including crop variety, soil fertility, rainfall, temperature, irrigation, fertilizer use, pests, and farming practices. Estimating production using experience or limited records alone can be difficult, especially when weather and growing conditions change. The proposed system uses historical agricultural data and artificial intelligence (AI) to estimate expected crop yield and support decisions about crop production.

1.2 Objectives of the Proposed System

The major objectives of the proposed system are:

1. To predict crop yield using relevant agricultural and environmental information.
2. To identify factors that may influence crop production.
3. To help farmers compare crop choices and make data-informed decisions.
4. To provide practical recommendations for improving farming practices and resource use.
5. To present predictions and analysis through an accessible software interface.

1.3 Scope of the System

The scope includes agricultural data entry or upload, data preparation, AI-based yield prediction, presentation of results, and generation of general optimization recommendations. Intended users may include farmers, agricultural experts, researchers, and agricultural organizations. The initial version may focus on crops and regions represented in its available dataset; support for additional crops, regions, and live data sources can be added later.

2. Identify Key Stakeholders and Challenges

2.1 Key Stakeholders

Stakeholder	Roles and Responsibilities
Farmers	Provide farm details and use predictions to inform crop selection, input planning, and farming decisions.
Agricultural experts / extension officers	Review whether the results and recommendations suit local agricultural conditions.
Agricultural researchers	Study agricultural patterns and help evaluate the usefulness of the model.
AI / data scientists	Prepare datasets, train prediction models, and evaluate model performance.
Software developers	Design, implement, test, and maintain the application and its modules.
System administrators	Manage user accounts, permissions, backups, system configuration, and availability.
Agricultural organizations / government departments	Use appropriate aggregated insights to support agricultural planning and advisory services.

2.2 Major Challenges

- Data quality and availability: agricultural datasets may contain missing, inconsistent, outdated, or inaccurate values.
- Regional differences: soil, climate, crop varieties, and farming practices differ across locations.
- Weather and environmental uncertainty: unexpected rainfall, drought, temperature changes, pests, and diseases affect actual yield.
- Accessibility and interpretation: some users may have limited internet access or digital experience.
- Model maintenance: the system requires ongoing validation and updates as new agricultural data becomes available.

3. Analyze the Current Approach

3.1 Existing System or Current Process

Crop planning and yield estimation commonly draw on farmers' experience, previous harvest records, local weather observations, advice from agricultural experts, and information from agricultural departments. These sources provide useful local knowledge, but information may be stored in different formats and may not always be analyzed together using a consistent predictive process.

3.2 Strengths of the Existing Approach

- Farmers can apply practical experience and knowledge of local conditions.
- Existing practices may require little advanced technological infrastructure.
- Historical records and local expert advice provide useful context.
- Human judgment can account for unusual or location-specific conditions.

3.3 Limitations of the Existing Approach

- Manual estimates may vary between individuals and can be subjective.
- Analyzing large datasets by hand can require significant time and effort.
- It may be difficult to identify relationships among several agricultural factors at once.
- Unexpected weather or changing farming conditions can make past experience less reliable.

3.4 Identified Gaps

1. A need to combine relevant agricultural factors in one consistent analysis process.
2. A need for repeatable yield estimates based on validated data and prediction methods.
3. A need to present results clearly for comparison and planning.
4. A need to explain model uncertainty and the limits of recommendations.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Proposed Software Solution

The proposed system is a software platform that collects agricultural information, validates and processes it, applies a trained AI model, and presents yield estimates and decision-support recommendations through a dashboard. Users may enter or upload crop type, location, soil characteristics, rainfall, temperature, irrigation, fertilizer use, and historical yield where available.

The system workflow can be represented as:

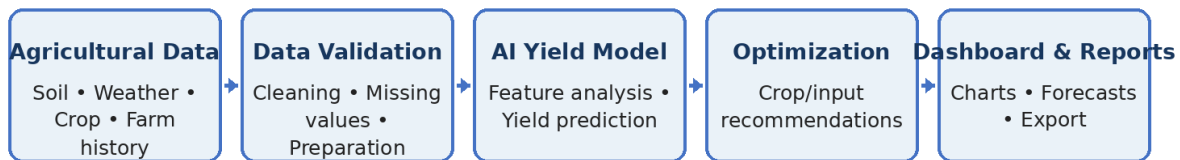
**Agricultural Data → Data Validation and Preprocessing → AI Yield Prediction → Optimization
Recommendations → Dashboard and Reports**

4.2 Major Functional Modules and Architecture Diagram

- User and Access Management Module – manages user registration, authentication, profiles, and access permissions.

- Agricultural Data Collection Module – allows users to enter farm information or upload relevant datasets.
- Data Preprocessing Module – checks missing values, inconsistent formats, and invalid entries, then prepares data for model input.
- AI-Based Yield Prediction Module – estimates crop yield from available agricultural parameters using a trained model.
- Crop Optimization Module – presents data-informed suggestions about crop choices and farming practices.
- Reports and Visualization Module – displays prediction results, trends, comparisons, and exportable summaries.

AI-Powered Crop Yield Prediction and Optimization



User input → preprocessing → trained model → recommendations → decision support

4.3 Functional Requirements

The system shall:

1. Allow authorized users to access the platform.
2. Accept agricultural data through forms or supported file uploads.
3. Validate and preprocess input data before prediction.
4. Generate crop-yield estimates using the selected AI model.
5. Display predictions and related visualizations in an understandable format.
6. Provide optimization recommendations when sufficient information is available.
7. Generate or export results in suitable report formats.

4.4 Non-Functional Requirements

- Security: protect user and agricultural data through authentication, authorization, and secure data handling.
- Reliability: handle invalid inputs and software failures appropriately and preserve stored information.
- Performance: process supported inputs and display results within an acceptable response time.
- Usability: provide clear forms, readable charts, and understandable explanations.
- Scalability: accommodate additional users, crops, regions, and datasets.
- Maintainability: use documented, modular components that can be tested and updated independently.

4.5 Application of Software Engineering Principles

- Modularity: separate data collection, preprocessing, prediction, recommendation, and reporting into independent components.
- Scalability: design the architecture to accommodate more users, crop types, locations, and data sources.
- Maintainability: use documentation, coding standards, version control, and independent module testing.
- Reliability: apply input validation, model evaluation, error handling, logging, and backup procedures.
- Usability: provide a simple interface, clear charts, and plain-language explanations.
- Security: use authentication, role-based access, protected data transfer, and appropriate handling of user information.

5. Justify the Chosen Methodology/Framework

5.1 Selected Methodology: Agile Scrum

Agile development using Scrum is proposed for this project. Work can be organized into short development cycles, or sprints, with a review of progress at the end of each cycle.

5.2 Justification for Selecting Agile Scrum

- Requirements may evolve after model testing and feedback from farmers or agricultural experts.
- A basic working version can be developed first and improved incrementally.
- Data preparation, model performance, and interface usability can be evaluated repeatedly.
- Issues can be identified early and addressed during development.

5.3 Proposed Sprint Structure

Sprint	Major Activities
Sprint 1	Requirement analysis, initial interface design, user management, and architecture.
Sprint 2	Agricultural data collection, validation, and database development.
Sprint 3	Data preprocessing and initial AI model development and evaluation.
Sprint 4	Yield prediction, optimization recommendations, and dashboard integration.
Sprint 5	Report generation, security checks, usability testing, and final evaluation.

5.4 Support for Development and Maintenance

Regular sprint reviews allow stakeholders to comment on the working system and clarify requirements. Testing during each sprint helps detect defects early. After deployment, the same iterative process can be used to fix issues, update datasets, validate revised models, improve usability, and introduce additional features.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Outcomes

The proposed system is expected to:

1. Provide AI-based estimates of crop yield using relevant agricultural and environmental data.
2. Support more informed crop selection, farm planning, and preparation for harvesting and storage.
3. Help identify agricultural factors associated with changes in crop production.
4. Offer data-informed recommendations for improving farming practices and using resources efficiently.
5. Make agricultural analysis more accessible through a clear interface and visual summaries.
6. Support comparison of historical information and predicted crop performance.
7. Help users plan agricultural inputs and activities more systematically.
8. Provide a basis for evaluating prediction quality using appropriate model-performance metrics.
9. Improve access to understandable agricultural insights for users with different levels of technical knowledge.
10. Create a foundation for future integration of live weather, sensor, satellite, and regional-language features.

6.2 How the Proposed Solution Addresses the Challenges

The system organizes multiple agricultural factors in one workflow instead of relying only on separate records or manual estimation. Data validation can flag missing or inconsistent inputs, while model evaluation can help

users understand how well predictions perform on available test data. Visual explanations and clearly stated limitations can help users interpret results. Feedback from agricultural experts can be used to improve recommendations and adapt the system to local needs.

6.3 Risks and Implementation Challenges

Risk	Possible Mitigation
Poor-quality or incomplete data	Validate inputs, document data sources, and preprocess datasets carefully.
Uncertain AI predictions	Evaluate the model on suitable test data, communicate uncertainty, and avoid treating predictions as guarantees.
Differences between regions and crops	Use representative local data and validate the model for each intended use.
Unexpected environmental events	Use updated data where available and explain that external events can affect actual yield.
Limited user access or digital experience	Provide a simple interface, clear guidance, and consider mobile or multilingual access.
Model performance changes over time	Monitor results and periodically review and retrain the model when appropriate.

6.4 Limitations

- The system cannot guarantee the exact future crop yield.
- Prediction quality depends on the relevance, quantity, and accuracy of available data.
- Unexpected weather, pests, diseases, and other environmental events may affect actual production.
- Recommendations may need to be adapted to local soil conditions, crop varieties, and farming practices.
- Developing and maintaining the system requires suitable technical resources and periodic model evaluation.

6.5 Future Enhancements

1. Integrate real-time weather services and local weather forecasts.
2. Connect IoT-based soil sensors for more frequent collection of field data.
3. Use satellite imagery and remote-sensing data to support broader agricultural observations.
4. Develop a mobile application and regional-language interface for easier farmer access.
5. Extend the system to additional crops and geographical regions using appropriate data.
6. Add scenario comparison and alerts for important changes in environmental conditions.
7. Collaborate with agricultural advisory services and experts to review recommendations.

Conclusion

The AI-Powered Crop Yield Prediction and Optimization System combines agricultural data analysis, AI-based prediction, and accessible software tools to support farm planning. Its usefulness will depend on relevant data, careful model evaluation, local interpretation, and regular maintenance. An iterative development process and sound software engineering principles can help the system improve over time and accommodate additional agricultural data sources and user needs.